



DSA – Sec A

Lab – 10

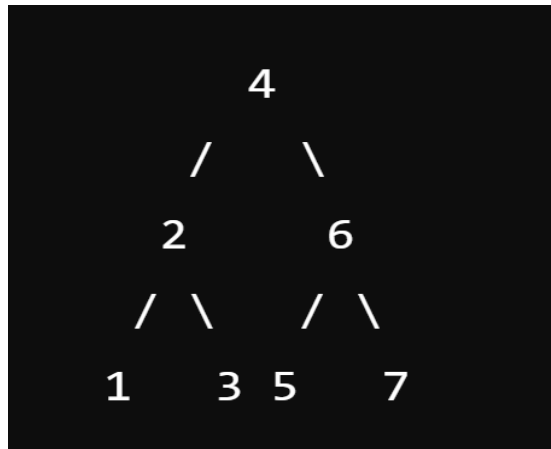
<u>Instructor</u>	<u>Mr. Shehryar</u>
<u>Lab Instructors</u>	• Muhammad Taha • Rimsha Aamir • Adeel Zafar

1. Sorted Array to Balanced BST

Given a sorted array. The task is to convert it into a Balanced Binary Search Tree (BST). Return the root of the BST.

For example:

With the array `[1, 2, 3, 4, 5, 6, 7]`, the balanced BST would look like this:



2. Binary Tree Traversals (InOrder, PreOrder, PostOrder, LevelOrder)

Implement the following traversals for a binary tree you build above:

- **InOrder Traversal:** Visit nodes in Left-Node-Right order.
 - Output: `[1, 2, 3, 4, 5, 6, 7]`
- **PreOrder Traversal:** Visit nodes in Node-Left-Right order.
 - Output: `[4, 2, 1, 3, 6, 5, 7]`
- **PostOrder Traversal:** Visit nodes in Left-Right-Node order.
 - Output: `[1, 3, 2, 5, 7, 6, 4]`
- **LevelOrder Traversal:** Visit nodes level by level.
 - Output: `[4, 2, 6, 1, 3, 5, 7]`

3. Check Equality of Two Binary Trees

- Implement functions to check if two trees are equal.
- **Function Signatures:**
 - `bool isValueEqual(Node* tree1, Node* tree2)` – Returns true if both trees have the same values.

- `bool isStructurallyEqual(Node* tree1, Node* tree2)` – Returns true if both trees have the same values and structure.

4. Check if a Binary Tree is BST or not

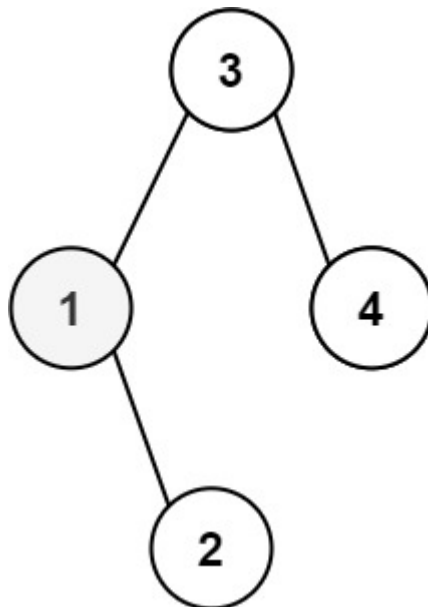
Given the root of a binary tree. Check whether it is a **Binary Search Tree** or **not**. A Binary Search Tree (BST) is a node-based binary tree data structure with the following properties.

- All keys in the left subtree are **smaller** than the root and all keys in the right subtree are **greater**.
- Both the left and right subtrees must also be binary search trees.
- Each key must be **distinct**.

5. Kth Smallest Element in a BST

Given the root of a binary search tree, and an integer *k*, return *the kth smallest value of all the values of the nodes in the tree*.

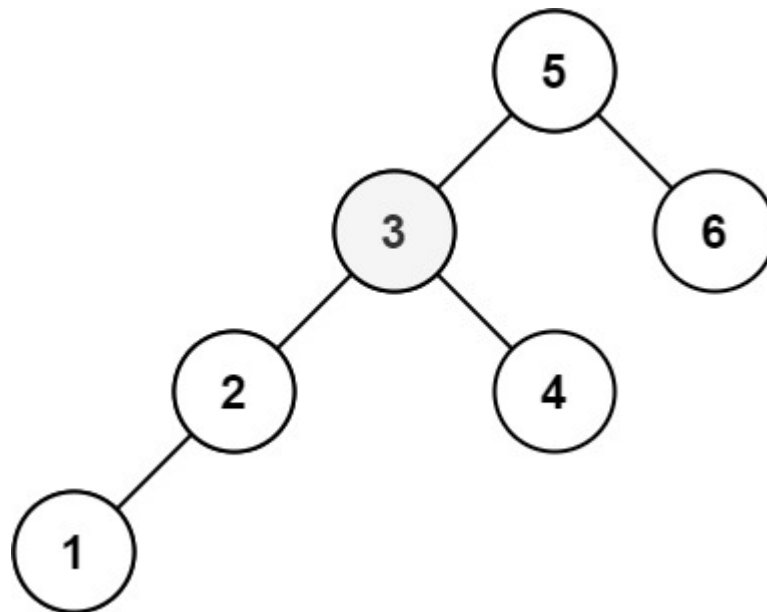
Example:1



Input: root = [3,1,4,null,2], k = 1

Output: 1

Example:2



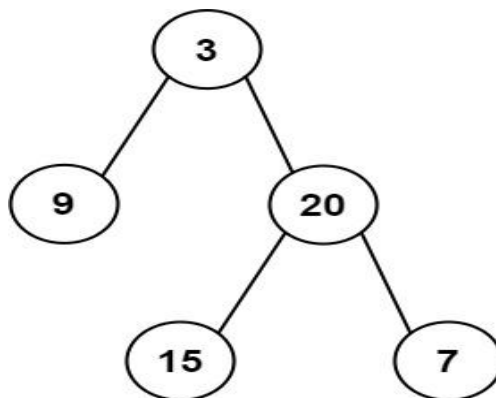
Input: root = [5,3,6,2,4,null,null,1], k = 3

Output: 3

6. Construct Binary Tree from Inorder and Postorder Traversal

Given two integer arrays inorder and postorder where inorder is the inorder traversal of a binary tree and postorder is the postorder traversal of the same tree, construct and return *the binary tree*.

Example:



Input: inorder = [9,3,15,20,7], postorder = [9,15,7,20,3]

Output: [3,9,20,null,null,15,7]

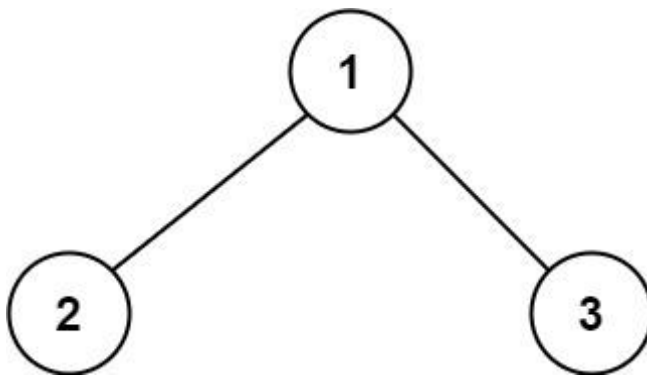
7. Binary Tree Maximum Path Sum

A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Note that the path does not need to pass through the root.

The path sum of a path is the sum of the node's values in the path.

Given the root of a binary tree, return the maximum path sum of any non-empty path.

Example 1:

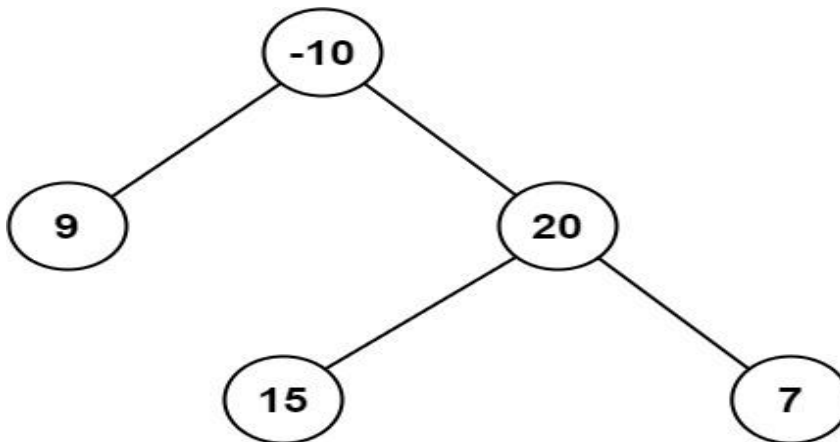


Input: root = [1,2,3]

Output: 6

Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of $2 + 1 + 3 = 6$.

Example 2:



Input: root = [-10,9,20,null,null,15,7]

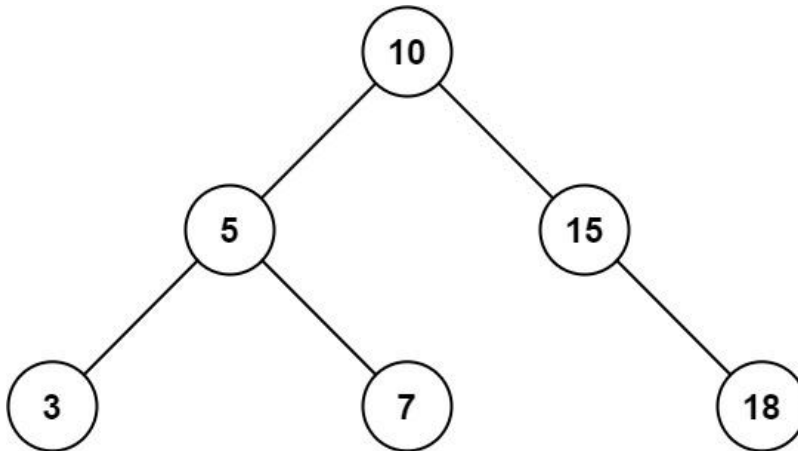
Output: 42

Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of $15 + 20 + 7 = 42$.

8. Range Sum of BST

Given the root node of a binary search tree and two integers low and high, return the sum of values of all nodes with a value in the inclusive range [low, high].

Example 1:

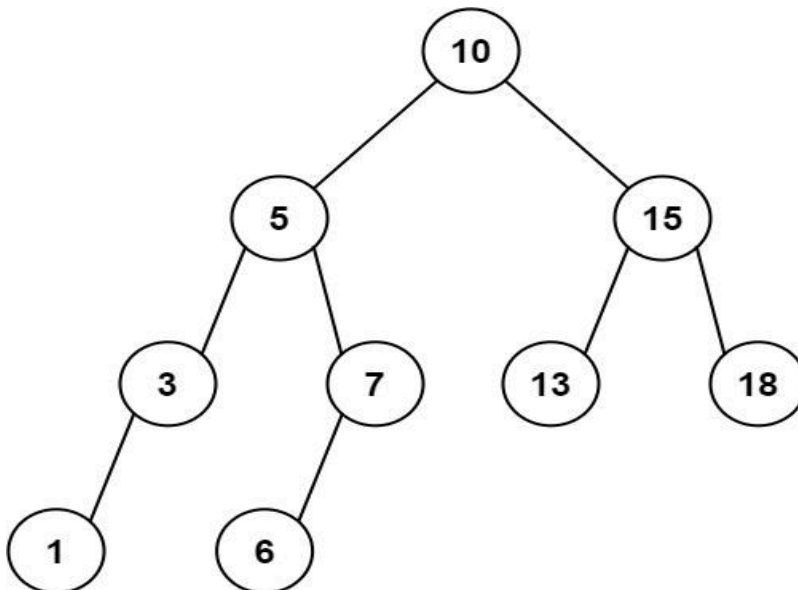


Input: root = [10,5,15,3,7,null,18], low = 7, high = 15

Output: 32

Explanation: Nodes 7, 10, and 15 are in the range [7, 15]. $7 + 10 + 15 = 32$.

Example 2:



Input: root = [10,5,15,3,7,13,18,1,null,6], low = 6, high = 10

Output: 23

Explanation: Nodes 6, 7, and 10 are in the range [6, 10]. $6 + 7 + 10 = 23$.

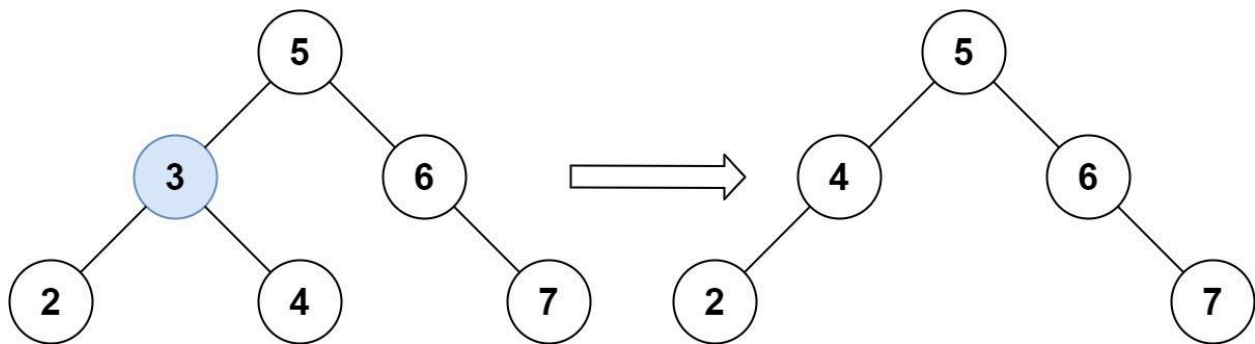
9. Delete Node in a BST

Given a root node reference of a BST and a key, delete the node with the given key in the BST. Return the root node reference (possibly updated) of the BST.

Basically, the deletion can be divided into two stages:

1. Search for a node to remove.
2. If the node is found, delete the node.

Example 1:



Input: root = [5,3,6,2,4,null,7], key = 3

Output: [5,4,6,2,null,null,7]

Explanation: Given key to delete is 3. So we find the node with value 3 and delete it.

One valid answer is [5,4,6,2,null,null,7], shown in the above BST.

Please notice that another valid answer is [5,2,6,null,4,null,7] and it's also accepted.

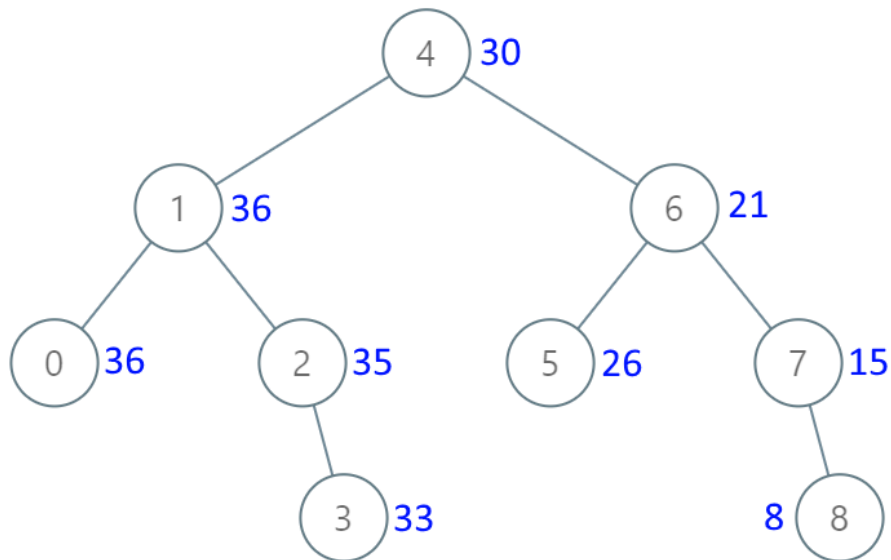
10. Convert BST to Greater Tree

Given the root of a Binary Search Tree (BST), convert it to a Greater Tree such that every key of the original BST is changed to the original key plus the sum of all keys greater than the original key in BST.

As a reminder, a binary search tree is a tree that satisfies these constraints:

- The left subtree of a node contains only nodes with keys less than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- Both the left and right subtrees must also be binary search trees.

Example:



Input: root = [4,1,6,0,2,5,7,null,null,null,3,null,null,null,8]

Output: [30,36,21,36,35,26,15,null,null,null,33,null,null,null,8]